# Scanner de documentos com transformação perspectiva em OpenCV

Gustavo Jakobi (GRR20221253)
Bruno Crestani (GRR20221240)
Departamento de Informática – Universidade Federal do Paraná (UFPR)
Visão Computacional e Percepção – Prof. Eduardo Todt
Código: `https://github.com/GJakobi/ta03-transformacao-perspectiva`

## Resumo

Este trabalho apresenta uma aplicação simples de transformação perspectiva: um scanner de documentos. A aplicação recebe a foto de uma folha tirada em ângulo, encontra os quatro cantos da folha, calcula a homografia que leva esses cantos a um retângulo e aplica `cv2.warpPerspective`. O resultado parece uma página escaneada. Testamos a aplicação em cinco fotos sintéticas com cantos conhecidos. Após um refinamento por ajuste de retas, o erro médio dos cantos ficou abaixo de 0,5 px em quatro casos e em 1,6 px no caso de perspectiva mais forte.

## 1 Introdução

Uma foto de um documento quase nunca é tirada exatamente de frente. A câmera fica inclinada, e a folha retangular aparece como um quadrilátero qualquer. Retas continuam retas, mas o paralelismo, os ângulos e as razões de distância mudam.

A folha é um objeto plano. Pontos de um plano e seus pontos na imagem estão relacionados por uma transformação projetiva (homografia) [1, 2, 4]. Portanto, uma única matriz $3 \times 3$ desfaz a perspectiva. A aplicação usa este fato para gerar uma imagem "escaneada" a partir de uma foto comum.

## 2 Fundamentação

Em coordenadas homogêneas, um ponto $(x, y)$ é escrito como $(x, y, 1)$. A homografia $\mathbf{H}$ leva um ponto $\mathbf{p}$ ao ponto $\mathbf{p}'$:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad u = \frac{x'}{w'},\ v = \frac{y'}{w'}. \tag{1}$$

A matriz tem 8 graus de liberdade. Cada correspondência $(x, y) \to (u, v)$ gera duas equações lineares:

$$ax + by + c - gxu - hyu = u, \tag{2}$$
$$dx + ey + f - gxv - hyv = v. \tag{3}$$

Com 4 pontos, sem 3 colineares, temos um sistema $8 \times 8$ com solução única. A função `cv2.getPerspectiveTransform` resolve este sistema.

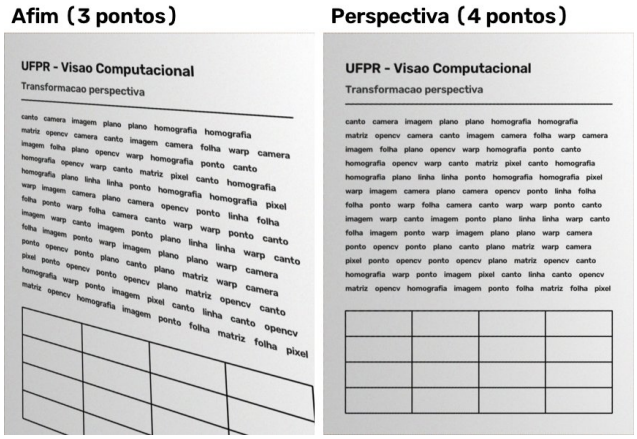



Figura 1: Retificação da mesma foto com uma transformação afim (3 cantos) e com uma homografia (4 cantos). A afim mantém as linhas paralelas da imagem e não corrige a convergência das bordas.

Uma transformação afim tem a última linha fixa em [0 0 1]. Ela preserva o paralelismo e fica definida por apenas 3 pontos. Por isso, ela não consegue levar um trapézio a um retângulo. A Figura 1 mostra este limite.

## 3 Método

A Figura 2 mostra as etapas da detecção.

**1. Detecção do quadrilátero.** A imagem é reduzida para 800 px de largura. Aplicamos desfoque gaussiano e um fechamento morfológico ($9 \times 9$). O fechamento remove o texto escuro e deixa a folha como uma região clara e uniforme. Em seguida aplicamos Canny, com limiares calculados a partir da mediana da imagem. Entre os 10 maiores contornos, escolhemos o primeiro que `approxPolyDP` reduz a 4 vértices convexos e que cobre mais de 10% da imagem.

**2. Refinamento dos cantos.** Os cantos da etapa 1 têm erro de alguns pixels. A redução da imagem e a aproximação poligonal causam este erro. Na resolução cheia, coletamos os pixels de borda numa faixa de $\pm 12$ px em torno de cada lado. Ajustamos uma reta a

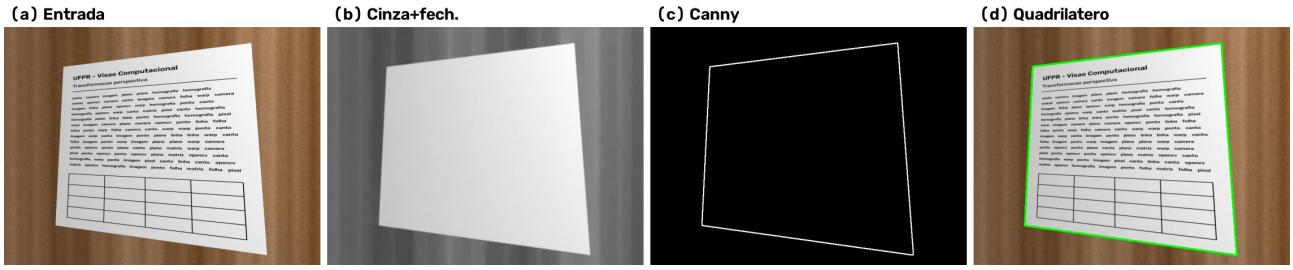



Figura 2: Etapas da detecção: (a) foto de entrada; (b) escala de cinza após o fechamento morfológico, que apaga o texto; (c) bordas de Canny; (d) maior contorno convexo com 4 vértices.

cada lado com `cv2.fitLine` (custo de Huber). Cada canto é a interseção de dois lados vizinhos, calculada pelo produto vetorial das retas em coordenadas homogêneas: $\mathbf{x} = \mathbf{l}_1 \times \mathbf{l}_2$.

**3. Ordenação e tamanho de saída.** Ordenamos os cantos como superior-esquerdo, superior-direito, inferior-direito e inferior-esquerdo. O menor valor de $x + y$ indica o superior-esquerdo, e o menor valor de $y - x$ indica o superior-direito. A largura de saída é o maior dos dois lados horizontais; a altura é o maior dos dois lados verticais.

**4. Homografia e warp.** Calculamos $\mathbf{H}$ com `getPerspectiveTransform` e aplicamos `warpPerspective` com interpolação cúbica [3]. O OpenCV faz o mapeamento inverso: para cada pixel de saída, ele calcula $\mathbf{H}^{-1}\mathbf{p}'$ e interpola a foto de entrada.

**5. Efeito de scanner.** Estimamos o fundo da página com dilatação e filtro de mediana. Dividimos a imagem por este fundo para remover sombras. Por fim, aplicamos `adaptiveThreshold` gaussiano.

**Modo manual.** Se a detecção falhar, a opção `--manual` abre uma janela. O usuário clica nos 4 cantos, e o restante do processo não muda.

O núcleo da aplicação cabe em poucas linhas:

```
cantos = detectar_cantos(img)
dst = np.float32([[0,0],[w-1,0],
                  [w-1,h-1],[0,h-1]])
H = cv2.getPerspectiveTransform(cantos, dst)
out = cv2.warpPerspective(img, H, (w, h))
scan = efeito_scanner(out)
```

# 4 Experimentos

Fotos reais não informam a posição exata dos cantos. Por isso, geramos fotos sintéticas: um documento de $850 \times 1100$ px é projetado sobre um fundo de madeira simulado, com uma homografia conhecida. Adicionamos iluminação desigual, ruído gaussiano, desfoque e compressão JPEG. Criamos cinco casos, do quase frontal até uma perspectiva forte.

Medimos três valores para cada caso:

- o erro médio dos cantos (px), antes e depois do refinamento;
- a correlação cruzada normalizada (NCC) entre o documento retificado e o original. NCC $= 1$ indica imagens iguais. A NCC não depende do brilho, então a iluminação desigual não altera a medida;
- a maior diferença entre a matriz do nosso sistema $8 \times 8$ e a matriz do OpenCV.

# 5 Resultados e discussão

A Tabela 1 e a Figura 3 mostram os resultados. A detecção encontrou a folha nos cinco casos. Sem refinamento, o erro médio dos cantos ficou em cerca de 2 px, porque a detecção usa a imagem reduzida pela metade. No caso de perspectiva forte, um canto ficou a 68 px da posição correta, e o documento saiu deformado (NCC negativa). O ajuste de retas reduziu o erro para menos de 0,5 px em quatro casos e para 1,6 px no caso mais difícil.

A NCC mostra que erros pequenos nos cantos têm efeito grande. Com 2 px de erro, o texto retificado fica deslocado em relação ao original, e a NCC cai de 0,92 para cerca de 0,7. Após o refinamento, a NCC fica perto do limite dado pelos cantos reais.

O caso de perspectiva forte continua o mais difícil. A borda superior da folha ocupa poucos pixels na foto. A homografia amplia esta região muitas vezes, e qualquer erro nos cantos também é ampliado. Mesmo com os cantos reais, esta região fica borrada, porque a foto não tem resolução suficiente ali.

Nosso sistema $8 \times 8$ e a função do OpenCV deram a mesma matriz. A diferença máxima ficou abaixo de $10^{-12}$, que é o erro de arredondamento numérico.

**Limitações.** A detecção automática supõe que a folha é a maior região clara com 4 lados e que ela contrasta com o fundo. Folha branca sobre mesa branca, cantos fora da foto ou folha dobrada fazem a detecção falhar. Nestes casos, use o modo manual. A homografia também supõe que a folha é plana; uma página curva de livro não é corrigida.

Tabela 1: Resultados nas fotos sintéticas. “Cantos reais” é a NCC obtida com os cantos verdadeiros e serve como limite superior.

| | Erro dos cantos (px) | | NCC com o original | | | |
|---|---|---|---|---|---|---|
| Caso | sem refino | com refino | sem refino | com refino | cantos reais | $\max \lvert H_{\text{nosso}} - H_{\text{cv}} \rvert$ |
| frontal_leve | 2.41 | 0.26 | 0.741 | 0.923 | 0.930 | 2e-13 |
| inclinada_lateral | 1.71 | 0.25 | 0.818 | 0.915 | 0.918 | 6e-14 |
| vista_de_baixo | 2.41 | 0.47 | 0.670 | 0.896 | 0.911 | 1e-13 |
| rotacionada | 2.21 | 0.18 | 0.703 | 0.919 | 0.922 | 6e-17 |
| forte_perspectiva | 21.09 | 1.56 | -0.057 | 0.583 | 0.879 | 5e-13 |

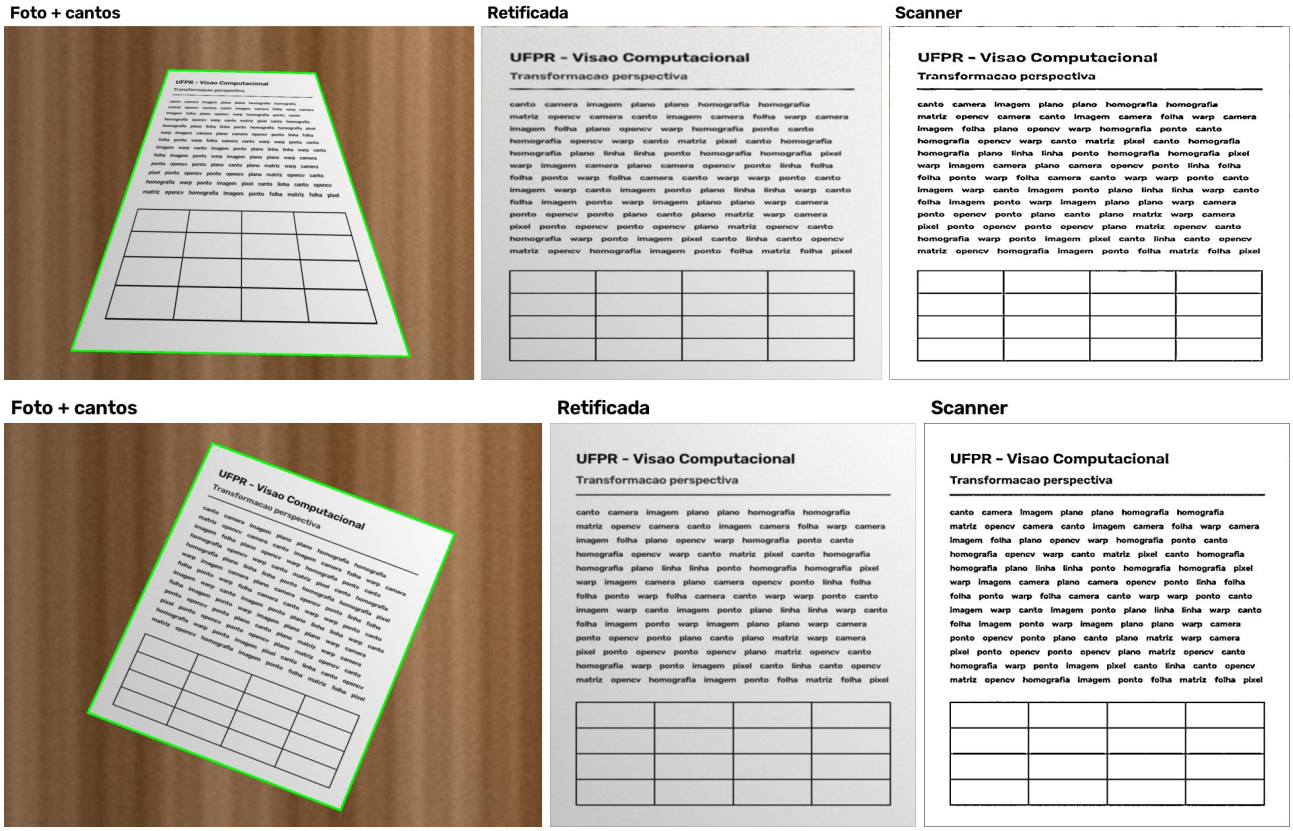



Figura 3: Dois casos de teste. Da esquerda para a direita: foto com os cantos detectados, documento retificado e saída com efeito de scanner.

## 6 Conclusão

Uma homografia calculada a partir de 4 pontos basta para remover a perspectiva de um objeto plano. A parte difícil da aplicação não é o warp, mas encontrar os 4 cantos com precisão. O refinamento por ajuste de retas reduziu o erro dos cantos para menos de 2 px em todos os casos e melhorou muito a qualidade do documento retificado. O código está disponível em `https://github.com/GJakobi/ta03-transformacao-perspectiva`.

## Referências


[1] R. Hartley e A. Zisserman. *Multiple View Geometry in Computer Vision*. 2. ed. Cambridge University Press, 2004.

[2] R. Szeliski. *Computer Vision: Algorithms and Applications*. 2. ed. Springer, 2022.

[3] OpenCV. *Geometric Transformations of Images*. `https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html`.

[4] E. Todt. *Transformations and warping*. Notas de aula, Visão Computacional e Percepção, UFPR, 2026.